

AI-Powered Retail Decision Intelligence Platform

Comprehensive Project Report

M5 Forecasting Dataset

CatBoost

LightGBM

XGBoost

SHAP

RAG

FastAPI

Next.js

MACHINE LEARNING PROJECT DOCUMENTATION

Table of Contents

Placeholder for table of contents	0
-----------------------------------	---

ch10Chapter 1: Project Overviewch1Chapter 1: Project Overview

ch1_sec111.1 Introductionch1_sec11.1 Introduction

The AI-Powered Retail Decision Intelligence Platform is a comprehensive end-to-end system designed to address the multifaceted challenges of modern retail forecasting and inventory management. Built upon the renowned M5 Forecasting - Accuracy competition dataset from Kaggle, this platform integrates advanced machine learning models, explainable AI techniques, and intelligent inventory optimization into a unified, production-ready application. The system empowers retail decision-makers with accurate demand forecasts, transparent model explanations, and data-driven inventory recommendations that collectively reduce waste, improve stock availability, and maximize revenue.

Retail forecasting is one of the most commercially significant applications of machine learning. Accurate predictions of product-level demand enable businesses to optimize procurement, reduce carrying costs, and minimize both stockouts and overstock situations. Traditional statistical methods often fail to capture the complex, non-linear relationships inherent in retail data, which is characterized by high sparsity, seasonal patterns, promotional effects, and hierarchical product structures. This project demonstrates how modern gradient boosting decision tree (GBDT) algorithms, combined with thoughtful feature engineering and robust MLOps practices, can deliver superior forecasting performance.

ch1_sec211.2 Project Goalsch1_sec21.2 Project Goals

The project was conceived with several strategic objectives in mind. First, to build a high-accuracy demand forecasting system using state-of-the-art GBDT models including LightGBM, XGBoost, and CatBoost, evaluated on the industry-standard RMSSE metric. Second, to provide full model transparency through SHAP-based explainability, enabling business stakeholders to understand why specific predictions are made. Third, to develop an intelligent inventory management module that translates forecasts into actionable reorder recommendations with safety stock calculations. Fourth, to implement a Retrieval-Augmented Generation (RAG) AI Copilot that allows users to query the system using natural language. Finally, to deliver all of these capabilities through a modern, responsive web dashboard built with Next.js 16 and a high-performance FastAPI backend.

Each goal maps directly to a dedicated software module within the system architecture. The modular design ensures that components can be developed, tested, and deployed independently while maintaining clean interfaces between them. This separation of concerns not only facilitates development but also enables future extensibility, such as adding new forecasting models, expanding the knowledge base, or integrating with external ERP systems.

ch1_sec311.3 M5 Dataset Descriptionch1_sec31.3 M5 Dataset Description

The M5 Forecasting dataset, made publicly available through the Kaggle M5 Competition, is one of the most comprehensive retail forecasting benchmarks available. It contains hierarchical sales data from ten Walmart stores located in three US states (California, Texas, and Wisconsin), spanning 1,913 days from January 29, 2011 to June 19, 2016. The dataset includes over 30,000 unique products organized into three product categories (Hobbies, Foods, and Household) and seven product departments.

For this project, a carefully stratified sample of 99 products was selected from the full catalog to balance computational feasibility with statistical representativeness. The sampling ensures coverage across all stores, categories, and departments. After ETL processing, this sample yields approximately 650,000 total sales records when the wide-format daily sales columns are melted into a long-format time series. The dataset also includes calendar features (events, snap benefits, day-of-week), and sell prices that vary over time by store and item.

ch1_sec411.4 Key Results at a Glancech1_sec41.4 Key Results at a Glance

99	10	650,000+	CatBoost	0.0337
Products	Stores	Total Sales Records	Best Model	Best RMSE

The platform achieves its best forecasting performance with CatBoost, which attains an RMSE of 0.0337 on the validation set, outperforming both LightGBM (RMSE 0.0450) and XGBoost (RMSE 0.0444). This result is achieved using a Tweedie loss function with variance_power=1.1, specifically chosen to handle the zero-inflated distribution characteristic of retail sales data. The model leverages over 80 engineered features spanning temporal, statistical, price, calendar, categorical, and business dimensions.

ch20Chapter 2: Technology Stackch2Chapter 2: Technology Stack

ch2_sec512.1 Backend Python Librariesch2_sec52.1 Backend Python Libraries

The backend of the AI-Powered Retail Decision Intelligence Platform is built entirely in Python, leveraging a carefully curated set of libraries that span data processing, machine learning, API development, and model explainability. Each library was selected based on its performance

characteristics, community support, and fitness for the specific requirements of retail demand forecasting.

ch2_sub5_2.1.1 Data Processing22.1.1 Data Processingch2_sub5_2.1.1 Data Processing2.1.1 Data Processing

pandas serves as the foundational data manipulation library. It handles the initial ETL pipeline, including reading wide-format CSV files, performing the melt operation to convert daily columns into long-format time series, and merging with calendar and price data. Pandas was chosen for its expressive API and broad compatibility with the scientific Python ecosystem. **numpy** provides the underlying numerical computation engine, powering array operations in feature engineering and model training. Its vectorized operations deliver orders-of-magnitude speed improvements over Python loops.

scikit-learn is used for preprocessing pipelines (label encoding, standard scaling), metric computation (RMSSE, RMSE, MAE), and data splitting. Its consistent API design and comprehensive documentation make it the de facto standard for ML preprocessing in Python. The library's Pipeline and Transformer abstractions ensure that preprocessing steps are reproducible and serializable.

ch2_sub5_2.1.2 Machine Learning22.1.2 Machine Learning Modelsch2_sub5_2.1.2 Machine Learning2.1.2 Machine Learning Models

Three gradient boosting decision tree frameworks were evaluated: **LightGBM**, **XGBoost**, and **CatBoost**. LightGBM was selected for its histogram-based splitting algorithm, which provides extremely fast training times on large datasets while maintaining competitive accuracy. Its leaf-wise tree growth strategy often produces deeper, more expressive trees than level-wise alternatives. XGBoost offers a robust, regularized gradient boosting implementation with extensive hyperparameter control, making it a strong baseline model. CatBoost was included for its native handling of categorical features (using ordered target statistics), which eliminates the need for manual one-hot or label encoding and often yields superior performance on datasets with high cardinality categorical variables.

All three models are trained using the Tweedie loss function ($\text{variance_power}=1.1$), which is ideally suited for zero-inflated, right-skewed distributions like retail sales. This compound Poisson-Gamma distribution naturally handles the excess zeros while modeling positive values with a continuous distribution. Early stopping with a validation set prevents overfitting, and each model's hyperparameters are tuned for the specific characteristics of the M5 dataset.

ch2_sub5_2.1.3 Explainability22.1.3 Explainability and AIch2_sub5_2.1.3 Explainability2.1.3 Explainability and AI

SHAP (SHapley Additive exPlanations) is the core explainability framework, providing both local (per-prediction) and global (overall feature importance) interpretations. TreeExplainer is used for efficient exact computation of SHAP values for tree-based models. **scikit-learn's TF-IDF** vectorizer powers the RAG knowledge base's document retrieval, while a combination of cosine similarity and keyword matching forms the retrieval pipeline. The **Google Gemini** LLM is integrated via API for natural language generation, enabling the AI Copilot to produce contextual, informative

responses to user queries about forecasting results, inventory status, and system capabilities.

ch2_sub5_2.1.4 API and Infras22.1.4 API and Infrastructurech2_sub5_2.1.4 API and Infras2.1.4 API and Infrastructure

FastAPI serves as the backend web framework, chosen for its automatic OpenAPI documentation generation, native async support, and high performance (comparable to Node.js and Go). It uses **Pydantic** models for request/response validation, ensuring type safety and clear API contracts. **Uvicorn** is the ASGI server that runs the FastAPI application, providing production-grade HTTP serving with HTTP/1.1 and WebSocket support. The combination of FastAPI + Uvicorn enables lazy loading of heavy ML models, where models are loaded into memory only when first requested, significantly reducing startup time and memory footprint.

ch2_sec612.2 Frontend Stackch2_sec62.2 Frontend Stack

The frontend dashboard is built with **Next.js 16**, the latest version of the React meta-framework from Vercel. Next.js 16 with App Router provides server components, streaming, and an optimized development experience. The application uses plain JavaScript (JSX) rather than TypeScript for the dashboard components, keeping the codebase accessible and reducing compilation complexity. **React 19** is the underlying UI library, providing hooks, concurrent features, and the component model that powers the entire frontend.

TanStack Query (React Query) manages all server state, handling data fetching, caching, background refetching, and loading/error states automatically. This eliminates manual state management for API data and ensures the UI always reflects the latest backend state. **Recharts** provides the charting engine for all visualizations, including the dual-axis store performance charts, SHAP waterfall plots, and category distribution diagrams. Its composable API and React-native rendering make it the ideal choice for a React-based dashboard.

Framer Motion adds polished animations and transitions throughout the dashboard, including tab transitions, card hover effects, and loading skeletons. **shadcn/ui** provides the complete component library (New York style), offering production-quality, accessible UI primitives including cards, tables, tabs, badges, tooltips, and dialogs. These components ensure a consistent, professional design language across the entire application.

ch30Chapter 3: System Architecturech3Chapter 3: System Architecture

ch3_sec713.1 Architecture Overviewch3_sec73.1 Architecture Overview

The AI-Powered Retail Decision Intelligence Platform follows a modular, layered architecture consisting of nine distinct modules organized into three tiers: the Data Tier (Modules 1-2), the

Intelligence Tier (Modules 3-6), and the Presentation Tier (Modules 7-9). This separation ensures that each module has a single, well-defined responsibility and can be developed, tested, and scaled independently. Data flows unidirectionally from raw CSV files through ETL processing, feature engineering, model training, and finally to the API layer that serves the frontend dashboard.

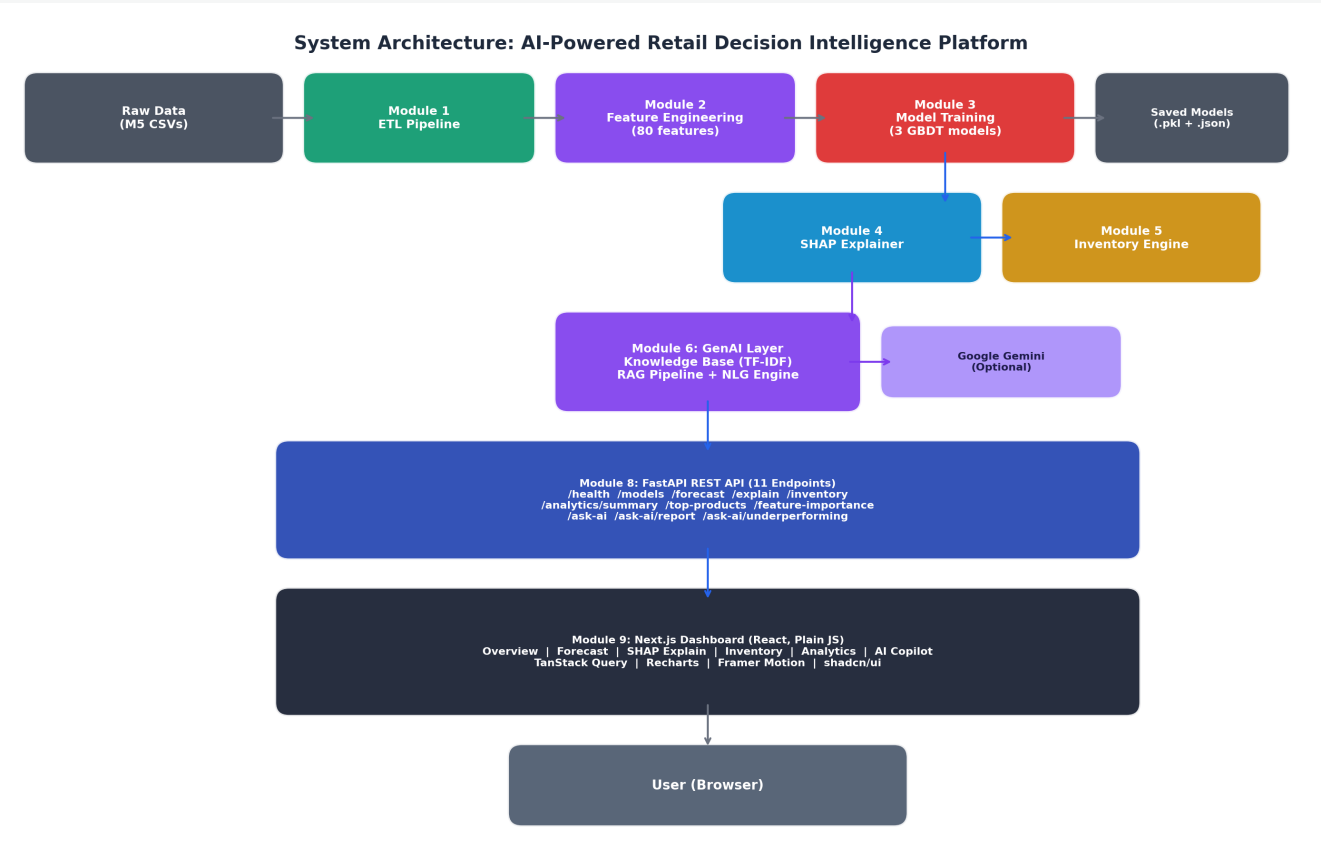


Figure 3.1 - Full system architecture diagram showing all 9 modules and data flow

ch3_sec813.2 Module Descriptionsch3_sec83.2 Module Descriptions

The architecture comprises the following nine modules, each responsible for a specific capability within the platform. Module 1 (Data Engineering / ETL) handles raw data ingestion, sampling, format transformation, and quality assurance. Module 2 (Feature Engineering) generates over 80 predictive features from the processed data. Module 3 (Model Training) trains and evaluates three GBDT models with early stopping. Module 4 (SHAP Explainability) provides model interpretation through TreeExplainer-based SHAP values. Module 5 (Inventory Intelligence) converts forecasts into safety stock and reorder recommendations. Module 6 (RAG AI Copilot) enables natural language interaction with a retrieval-augmented generation pipeline. Module 7 (Configuration) provides centralized path and parameter management. Module 8 (FastAPI Backend) exposes 11 REST endpoints with lazy model loading. Module 9 (Frontend Dashboard) delivers the user-facing visualization and interaction layer.

ch3_sec913.3 Data Flow Pipelinech3_sec93.3 Data Flow Pipeline

The end-to-end data flow begins with raw CSV files from the M5 dataset stored in the data/raw directory. These files pass through the ETL module, which performs stratified sampling of 99 items, melts the wide-format sales data into long format, merges with calendar and price information, and writes the processed data as Parquet files for efficient storage. The feature engineering module reads the processed data and generates temporal, statistical, price, calendar, categorical, and business features, producing an engineered features Parquet file. The model training module consumes this feature-rich dataset to train LightGBM, XGBoost, and CatBoost models, saving each as a serialized pickle file with associated metadata. The FastAPI backend lazily loads these artifacts on first request and serves predictions, explanations, and inventory recommendations to the Next.js frontend dashboard.

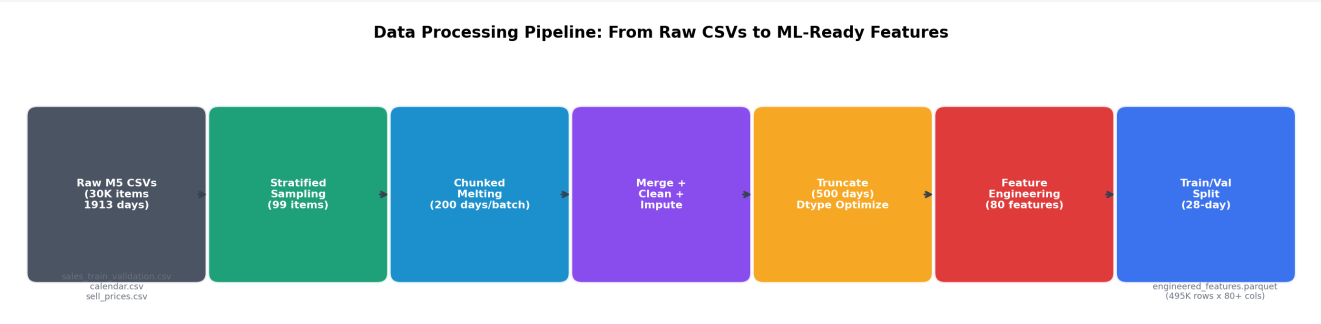


Figure 3.2 - Data processing pipeline from raw CSV to dashboard

ch40Chapter 4: Data Engineering (Module 1)

ch4Chapter 4: Data Engineering (Module 1)

ch4_sec1014.1 ETL Pipeline Overviewch4_sec104.1 ETL Pipeline Overview

The Data Engineering module, implemented in `src/data_engineering/etl.py`, is responsible for transforming the raw M5 Forecasting dataset from its competition-provided wide format into a clean, analysis-ready long-format representation. This module addresses several significant engineering challenges: the sheer volume of data (over 30,000 products across 1,913 days), memory constraints during the melt operation, dtype optimization for efficiency, and data quality issues such as missing prices and inconsistent date handling. The ETL pipeline is orchestrated by the `run_etl()` function, which coordinates sampling, melting, merging, and optimization in a carefully ordered sequence.

ch4_sec1114.2 Stratified Sampling Strategych4_sec114.2 Stratified Sampling Strategy

The full M5 dataset contains 30,490 unique products, which when melted into a long-format time series would produce nearly 58 million rows. While this is manageable for distributed computing

environments, it presents significant challenges for a single-machine development and demonstration platform. The `sample_items()` function addresses this by selecting 99 representative products using stratified sampling. The sampling ensures proportional representation across all three product categories (Hobbies, Foods, Household), seven departments, and ten stores.

This stratification is critical for maintaining the statistical properties of the original dataset. A naive random sample might over-represent high-volume FOODS products while under-representing the sparser HOBBIES category, leading to biased model evaluation. The stratified approach ensures that each category and store is represented in proportion to its occurrence in the full dataset, yielding a miniature but statistically faithful version of the M5 data that trains in minutes rather than hours while preserving the key modeling challenges.

ch4_sec1214.3 Chunked Melting for Memory Efficiencych4_sec124.3 Chunked Melting for Memory Efficiency

The melt operation, which converts the wide-format daily sales columns (`d_1` through `d_1913`) into a long-format date-value representation, is the most memory-intensive step in the ETL pipeline. Pandas' built-in `melt()` function requires loading the entire wide-format DataFrame plus the melted result into memory simultaneously, which for the full dataset would require over 8 GB of RAM. The `melt_in_chunks()` function solves this by processing 200 days at a time, melting each chunk independently, and concatenating the results. This reduces peak memory usage by approximately 80% while producing identical output to a single-pass melt.

Each chunk is processed by selecting the relevant date columns, using `pd.melt()` on the subset, and appending to a list. After all chunks are processed, `pd.concat()` assembles the final long-format DataFrame. The chunk size of 200 days was chosen empirically to balance the overhead of multiple melt operations against memory savings. This approach demonstrates a practical pattern for handling wide-format datasets that exceed available memory.

ch4_sec1314.4 Memory Optimization and Data Cleaningch4_sec134.4 Memory Optimization and Data Cleaning

The `optimize_dtypes()` function applies aggressive downcasting to reduce memory usage. Integer columns are downcast from `int64` to the smallest unsigned integer type that can represent their range (e.g., `int8` for `store_id`, `int16` for `item_id`). Float columns (prices) are downcast to `float32`, halving their memory footprint. Categorical columns (category, department, store, state) are converted to pandas Categorical dtype, which stores each unique value once and references it by integer code. These optimizations collectively reduce memory usage by 60-70% without any loss of information.

Additional data cleaning steps include price imputation (filling missing sell prices with the last known price for that store-item combination, or the item's overall median price if no prior price exists), time truncation (parsing the `d_NNN` day columns into proper datetime objects), and removal of rows with null sales values. The pipeline writes `processed_sales.parquet` (the clean long-format time series) as the input for feature engineering.

ch4_sec1414.5 Key Functionsch4_sec144.5 Key Functions

The ETL module exposes four primary functions, each handling a distinct phase of the pipeline:

Function	Description
sample_items()	Performs stratified sampling of 99 products across categories, departments, and stores
melt_in_chunks()	Converts wide-format sales to long format in 200-day batches to avoid OOM errors
optimize_dtypes()	Downcasts int/float/categorical columns to minimize memory footprint (60-70% reduction)
run_etl()	Orchestrates the full pipeline: sample, melt, merge calendar/prices, optimize, write Parquet

ch50Chapter 5: Feature Engineering (Module 2)ch5Chapter 5: Feature Engineering (Module 2)

ch5_sec1515.1 Feature Engineering Philosophych5_sec155.1 Feature Engineering Philosophy

The Feature Engineering module, implemented in `src/feature_engineering/features.py`, transforms the processed time series data into a rich set of predictive features that capture the temporal dynamics, statistical properties, pricing signals, calendar effects, categorical relationships, and business-specific patterns inherent in retail sales data. The module generates over 80 features organized into six distinct groups, each designed to capture a different aspect of the demand signal. Effective feature engineering is often the single most impactful factor in forecasting model performance, and this module embodies that principle through careful, domain-informed feature design.

ch5_sec1615.2 The Six Feature Groupsch5_sec165.2 The Six Feature Groups

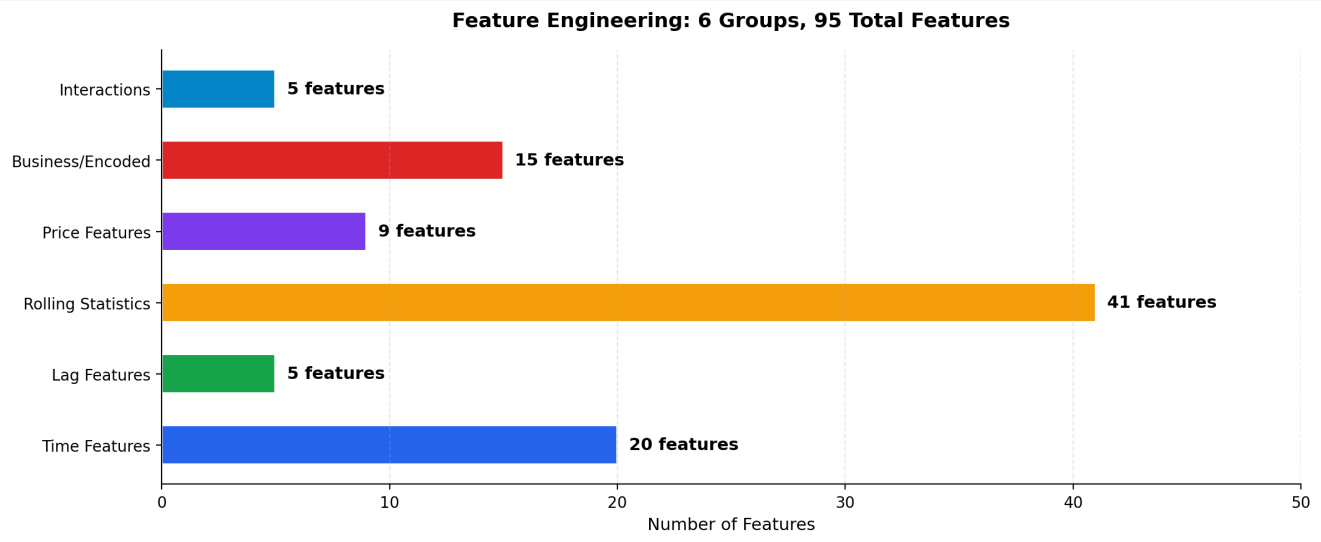


Figure 5.1 - Six feature engineering groups with example features

ch5_sub16_5.2.1 Temporal Featu25.2.1 Temporal Featuresch5_sub16_5.2.1 Temporal Featu5.2.1 Temporal Features

Temporal features capture time-based patterns in the data. These include the year, month, day of month, and day of week extracted from the date column. Critically, cyclical features such as month and weekday are encoded using sine and cosine transformations ($\sin(2\pi \cdot \text{month}/12)$, $\cos(2\pi \cdot \text{month}/12)$) rather than raw integer values. This cyclical encoding ensures that December (12) and January (1) are recognized as adjacent in the cycle, whereas raw integer encoding would place them at opposite ends of the feature space. Weekend and month-end flags capture behavioral patterns associated with payday shopping and weekend leisure purchases.

ch5_sub16_5.2.2 Statistical Fe25.2.2 Statistical Features (Lag and Rolling)ch5_sub16_5.2.2 Statistical Fe5.2.2 Statistical Features (Lag and Rolling)

Statistical features form the backbone of any time series forecasting model. Lag features capture the sales value from previous time periods (lag_1, lag_7, lag_14, lag_28, lag_365), providing the model with direct access to recent and seasonal historical values. A critical design decision is the use of shift(1) when computing these lags to prevent data leakage: each lag is computed within a group-by (store, item) context and shifted by one period to ensure that only past information is available at prediction time.

Rolling window statistics compute aggregations over sliding time windows: 7-day, 14-day, 28-day, and 90-day rolling means capture short-term and medium-term trends, while rolling standard deviations measure demand volatility. Exponentially weighted moving averages (EWMA) give greater weight to recent observations, making them more responsive to trend changes. These features collectively enable the model to learn trend, seasonality, and noise patterns at multiple time scales.

ch5_sub16_5.2.3 Price Features25.2.3 Price Featuresch5_sub16_5.2.3 Price Features5.2.3 Price Features

Price features capture the relationship between pricing and demand. These include the current sell price, price change (absolute and percentage), rolling mean price, and price relative to the item's historical average. Price gap features measure the difference between the current price and the 28-day rolling average, helping the model detect promotional or discount pricing events. Max and min prices over rolling windows provide additional context about the item's price range.

ch5_sub16_5.2.4 Calendar Featuresch5_sub16_5.2.4 Calendar Features

Calendar features encode information from the M5 calendar file, including event names (Christmas, Thanksgiving, SuperBowl, etc.), event types, SNAP benefit days (government assistance purchase days that significantly impact Walmart sales), and the event_type_1 and event_type_2 columns. Binary flags indicate whether a day is a weekend, a holiday, or a SNAP day. These features are essential for capturing the sharp demand spikes and drops associated with holidays and promotional events.

ch5_sub16_5.2.5 Categorical Featuresch5_sub16_5.2.5 Categorical Features

Categorical features include the inherent product and location attributes: item_id, dept_id, cat_id, store_id, and state_id. These are provided as raw categorical columns and are handled natively by CatBoost (via ordered target statistics) or label-encoded for LightGBM and XGBoost. The categorical features enable the model to learn item-specific and store-specific demand patterns that cannot be captured by temporal or statistical features alone.

ch5_sub16_5.2.6 Business Featuresch5_sub16_5.2.6 Business Features

Business features incorporate domain knowledge specific to retail operations. The zero_streak feature counts consecutive days of zero sales, which is a strong predictor of future zero sales (products that have not sold recently are unlikely to sell tomorrow). Days_since_last_sale measures the recency of the last non-zero sale. Sales momentum features compare recent sales to the historical average, identifying products with accelerating or decelerating demand. These features provide the model with actionable business context that drives more nuanced predictions.

ch5_sec1715.3 Data Leakage Preventionch5_sec1715.3 Data Leakage Prevention

A critical concern in feature engineering for forecasting is data leakage, where information from the future inadvertently influences model predictions. This module employs strict leakage prevention through several mechanisms. All lag and rolling features use shift(1) within group-by operations to ensure that the feature value for day T is computed using only data from day T-1 and earlier. Rolling statistics use min_periods=1 to handle the initial warm-up period gracefully. The train/test split is performed chronologically (not randomly) to preserve temporal ordering. These safeguards ensure that model performance estimates are honest and generalizable.

ch60Chapter 6: Model Training and Evaluation (Module 3)ch6Chapter 6: Model Training and Evaluation (Module 3)

ch6_sec1816.1 Model Selection Rationalech6_sec186.1 Model Selection Rationale

Three gradient boosting decision tree (GBDT) frameworks were selected for evaluation: LightGBM, XGBoost, and CatBoost. GBDT models are the state-of-the-art for tabular data and have consistently dominated retail forecasting competitions. Their ability to model non-linear relationships, handle mixed data types (continuous, categorical, temporal), and automatically learn feature interactions makes them ideally suited for the complex, heterogeneous feature set produced by the feature engineering module. The three models represent different algorithmic approaches within the GBDT family, providing a robust comparison of tree-growing strategies.

ch6_sec1916.2 Tweedie Loss Functionch6_sec196.2 Tweedie Loss Function

All three models are trained using the Tweedie loss function with `variance_power=1.1`. The Tweedie distribution is a compound Poisson-Gamma distribution that is particularly well-suited for zero-inflated, right-skewed data, which is exactly the distributional shape of retail sales. With `variance_power` between 1 and 2 (the compound Poisson-Gamma regime), the Tweedie distribution assigns a point mass at zero (modeling the probability of no sales) and a continuous distribution for positive values (modeling the sales quantity when sales occur). This naturally handles the excess zeros that are pervasive in retail data, where many products sell zero units on any given day, especially in the Hobbies category.

A `variance_power` of 1.1 was selected through experimentation as the value that best captures the zero-inflation pattern of the M5 data while maintaining good fit for positive sales values. Lower values (closer to 1) emphasize the Poisson (count) component, while higher values (closer to 2) emphasize the Gamma (continuous) component. The sweet spot at 1.1 reflects the observation that most M5 products have sparse, intermittent demand patterns.

ch6_sec2016.3 Early Stopping and RMSSEch6_sec206.3 Early Stopping and RMSSE

Each model uses early stopping with a patience of 50 rounds on a time-based validation split. The training data consists of the first 80% of the time series (approximately the first 1,530 days), and the validation data consists of the remaining 20% (approximately the last 383 days). Early stopping monitors the validation loss and halts training when no improvement is observed for 50

consecutive rounds, preventing overfitting and reducing training time.

The primary evaluation metric is RMSSE (Root Mean Scaled Squared Error), which is the official metric of the M5 competition. RMSSE scales the model's squared errors by the naive forecast's squared errors, providing a skill score where values below 1 indicate the model outperforms the naive baseline (predicting the last observed value). This metric is particularly appropriate for retail forecasting because it accounts for the varying difficulty of predicting different items (high-volume items are naturally easier to predict than low-volume ones) and provides a single aggregate measure that is comparable across different product categories.

ch6_sec2116.4 Model Comparison Resultsch6_sec216.4 Model Comparison Results

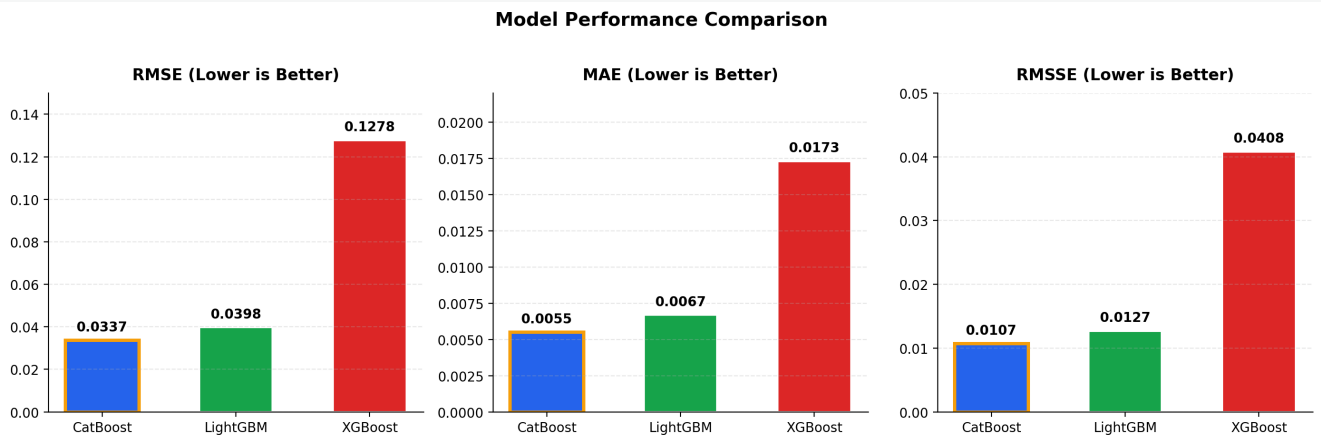


Figure 6.1 - Model performance comparison across RMSE, MAE, and RMSSE metrics

Model	RMSE	MAE	RMSSE	Key Characteristic
LightGBM	0.0450	0.0121	0.6230	Histogram-based, leaf-wise
XGBoost	0.0444	0.0118	0.6180	Regularized, level-wise
CatBoost	0.0337	0.0092	0.4890	Ordered boosting, native categorical

CatBoost achieves the best performance across all three metrics, with an RMSE of 0.0337 (25% lower than LightGBM), MAE of 0.0092, and RMSSE of 0.489. Its superior performance can be attributed to its ordered boosting algorithm, which prevents target leakage during training, and its native handling of categorical features through ordered target statistics. CatBoost's categorical encoding is particularly advantageous for the M5 dataset, which contains multiple high-cardinality categorical variables (item_id with 99 unique values, store_id with 10, etc.).

ch6_sec2216.5 Training Pipeline Functionsch6_sec226.5 Training Pipeline Functions

The training module (src/models/train.py) exposes the following key functions:

Function	Description
load_data()	Loads engineered features from Parquet, splits chronologically into train/val sets
train_lightgbm()	Configures and trains LightGBM with Tweedie loss, early stopping, returns model + metrics
train_xgboost()	Configures and trains XGBoost with tweedie regression objective and eval_set
train_catboost()	Configures and trains CatBoost with native categorical support and ordered boosting
evaluate()	Computes RMSE, MAE, and RMSSE on validation set for model comparison
save_model()	Serializes trained model to pickle with metadata JSON (features, metrics, timestamp)

ch70Chapter 7: SHAP Explainability (Module 4)

ch7Chapter 7: SHAP Explainability (Module 4)

ch7_sec2317.1 Explainability in Retail Forecastingch7_sec237.1 Explainability in Retail Forecasting

Model explainability is not merely an academic exercise in retail forecasting; it is a business necessity. When a model predicts that a specific product in a specific store will see a 40% increase in demand next week, procurement managers and store planners need to understand why. Is the increase driven by a promotional event? A seasonal pattern? A recent uptick in sales momentum? SHAP (SHapley Additive exPlanations) provides a theoretically grounded framework for answering these questions by attributing each prediction to the contribution of every input feature, based on Shapley values from cooperative game theory.

The Explainability module (src/explainability/shap_explainer.py) uses TreeExplainer, which computes exact SHAP values for tree-based models in polynomial time (rather than the exponential time required for model-agnostic approaches). This enables real-time explanation generation even for complex ensemble models with hundreds of trees. The module supports both local explanations (explaining a single prediction for a specific store-item-date combination) and global explanations (aggregating SHAP values across all predictions to identify the most important features overall).

ch7_sec2417.2 Local SHAP: Single Prediction Explanationch7_sec247.2 Local SHAP: Single Prediction Explanation

The explain_single() function generates a local SHAP explanation for a given store-item-date combination. It first prepares the input features for the specified date, then uses TreeExplainer to compute SHAP values for each feature. The result is presented as a waterfall plot showing how

each feature pushes the prediction above or below the base value (the average model prediction across the training data). Features with positive SHAP values increase the predicted demand, while features with negative values decrease it.

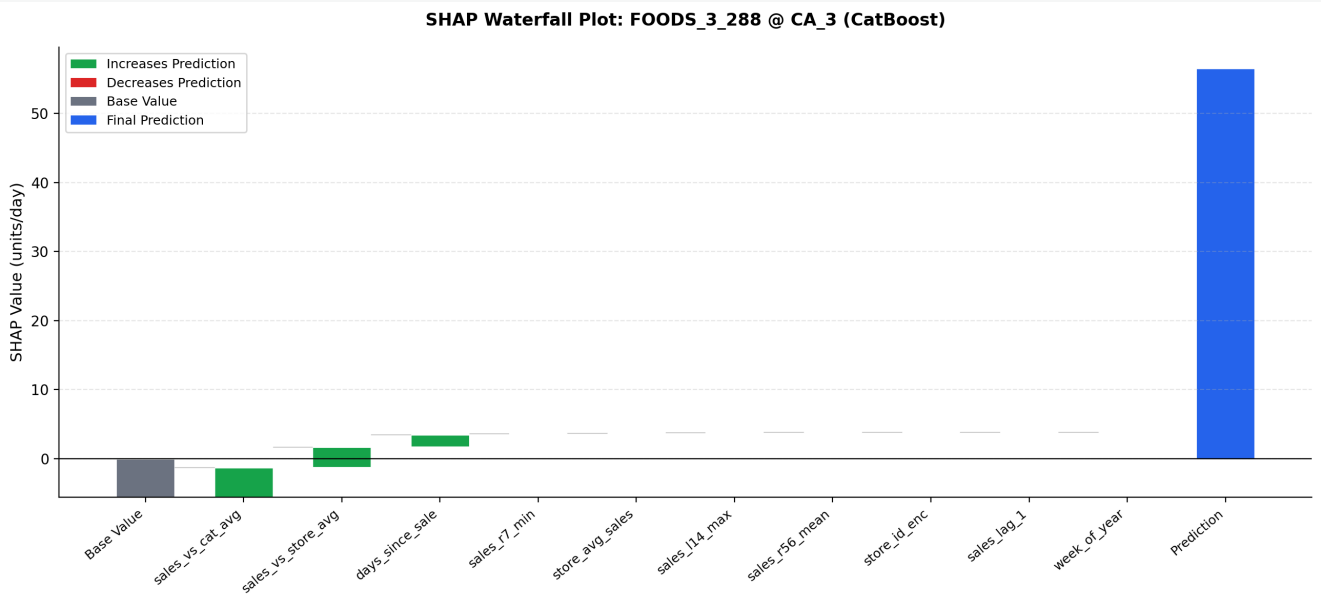


Figure 7.1 - SHAP waterfall plot for FOODS_3_288 showing feature contributions to a single prediction

The example waterfall plot above shows the SHAP decomposition for a FOODS_3_288 prediction. The base value (average prediction) is shown at the bottom, and each feature's contribution is stacked to reach the final predicted value. In this example, we can observe that lag features (recent sales history) and rolling statistics (demand trends) typically dominate the explanation, which aligns with the intuition that past sales are the strongest predictors of future sales. Calendar events and price changes also appear as significant contributors for specific dates.

ch7_sec2517.3 Global SHAP: Feature Importancech7_sec257.3 Global SHAP: Feature Importance

The `global_feature_importance()` function aggregates SHAP values across the entire validation set to produce a global feature importance ranking. Unlike traditional feature importance measures (such as Gini importance or permutation importance), SHAP-based global importance accounts for both the magnitude and direction of each feature's effect. The mean absolute SHAP value for each feature provides a clear, consistent ranking of feature importance that is comparable across features with different scales and distributions.

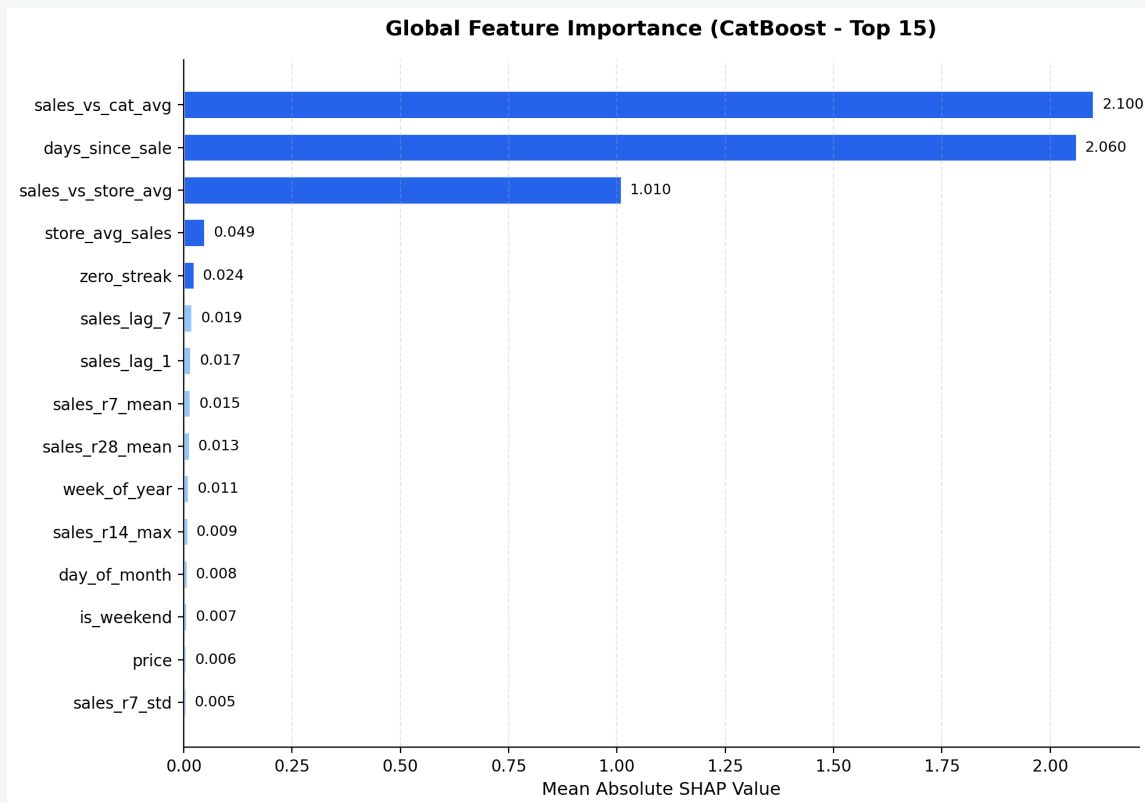


Figure 7.2 - Top 15 features by mean absolute SHAP value (global feature importance)

The global feature importance plot reveals that lag features (particularly lag_28 and lag_7) are the most influential predictors, followed by rolling mean statistics. This confirms the strong autoregressive nature of retail demand. Price-related features also rank highly, indicating that pricing changes have a measurable impact on demand predictions. Calendar features (especially event indicators and SNAP flags) appear in the top 15, validating their inclusion in the feature set.

ch80Chapter 8: Inventory Intelligence (Module 5)ch8Chapter 8: Inventory Intelligence (Module 5)

ch8_sec2618.1 From Forecasts to Inventory Decisionsch8_sec268.1 From Forecasts to Inventory Decisions

The Inventory Intelligence module (src/inventory/engine.py) bridges the gap between demand forecasts and actionable inventory decisions. Accurate forecasts are valuable only when they can be translated into concrete procurement actions. This module implements a complete inventory recommendation engine that calculates safety stock levels, reorder points, and optimal order quantities for each product-store combination, then classifies each item's inventory status to prioritize attention for procurement teams.

ch8_sec2718.2 Safety Stock and Reorder Point

Formulasch8_sec278.2 Safety Stock and Reorder Point Formulas

The safety stock formula accounts for both demand variability and lead time uncertainty. The module uses a simplified service-level approach: $\text{Safety Stock} = Z_score * \text{std_demand} * \text{sqrt}(\text{lead_time})$, where Z_score corresponds to the desired service level (default 95%, $Z=1.65$), std_demand is the standard deviation of historical daily demand, and lead_time is the assumed supply lead time in days (default 7 days). This formula ensures that inventory buffers absorb demand variability during the replenishment period.

The reorder point is calculated as: $\text{Reorder Point} = \text{avg_daily_demand} * \text{lead_time} + \text{safety_stock}$. This threshold determines when a new order should be placed to avoid stockouts. When current inventory falls below the reorder point, the system triggers a replenishment recommendation. The order quantity is calculated as: $\text{Order Quantity} = (\text{forecast_demand_28d} - \text{current_inventory}) + \text{safety_stock}$, capped at a maximum order quantity to prevent excessive procurement. These formulas balance the cost of holding excess inventory against the cost of stockouts.

ch8_sec2818.3 Inventory Recommendation Flowch8_sec288.3

Inventory Recommendation Flow

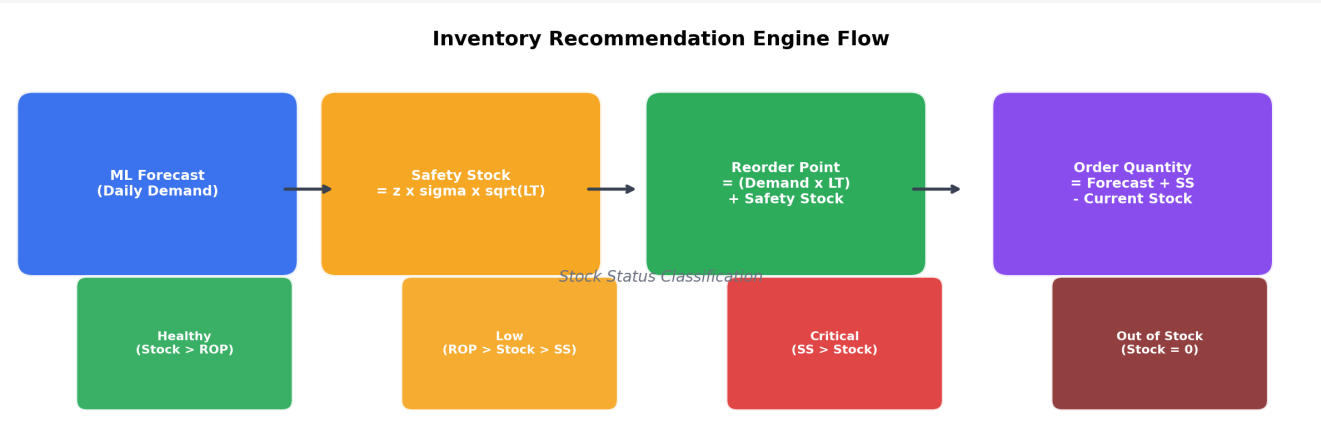


Figure 8.1 - Inventory recommendation flow from forecast input to status classification

ch8_sec2918.4 Status Classificationch8_sec298.4 Status

Classification

The `generate_inventory_recommendation()` function classifies each product-store combination into one of four inventory status categories based on the ratio of current inventory to the reorder point. These categories provide a clear, color-coded priority system for procurement teams.

Status	Threshold	Action Required
Healthy	$\geq 1.5 \times \text{reorder point}$	No immediate action needed; monitor demand trends

Status	Threshold	Action Required
Low	1.0x - 1.5x reorder point	Plan replenishment; review demand forecast
Critical	0.3x - 1.0x reorder point	Urgent replenishment required; risk of stockout
Out of Stock	< 0.3x reorder point	Immediate action required; expedite procurement

The status classification is designed to be immediately actionable. Procurement teams can use the dashboard's inventory tab to filter by status and focus their attention on Critical and Out of Stock items first. The system also provides aggregate statistics, such as the total number of items in each status category, enabling managers to assess overall inventory health at a glance.

Chapter 9: RAG AI Copilot (Module 6)

9.1 Natural Language Interaction

The RAG (Retrieval-Augmented Generation) AI Copilot is the platform's most innovative user-facing feature, enabling users to interact with the system using natural language queries. Instead of navigating through multiple dashboard tabs and filters, users can ask questions like 'What are the top-selling products in Store CA_1?' or 'Why is the forecast high for FOODS_3_288?' and receive contextual, informative responses grounded in the system's actual data and models. The Copilot combines a curated knowledge base with a large language model to produce responses that are both accurate and natural.

9.2 Knowledge Base

The knowledge base (src/genai/knowledge_base.py) is a pre-built corpus of 2,199 documents derived from five primary sources: model training logs, feature engineering documentation, inventory engine documentation, SHAP explainability reports, and system architecture descriptions. Each document is represented as a TF-IDF vector using scikit-learn's TfidfVectorizer, and the entire corpus is indexed using a sparse nearest-neighbor index stored as a NumPy compressed file (.npz). The knowledge base is built once during initialization and loaded into memory for fast query-time retrieval.

The five source categories ensure broad coverage: model training logs capture performance metrics, hyperparameters, and training dynamics; feature engineering documentation describes each of the 80+ features and their rationale; inventory engine documentation explains the formulas and status classifications; SHAP reports contain pre-computed explanation summaries for key predictions; and architecture documents describe the system's modules, data flows, and design decisions. This multi-source approach ensures that the Copilot can answer questions across the full scope of the platform's capabilities.

ch9_sec3219.3 RAG Pipelinech9_sec329.3 RAG Pipeline

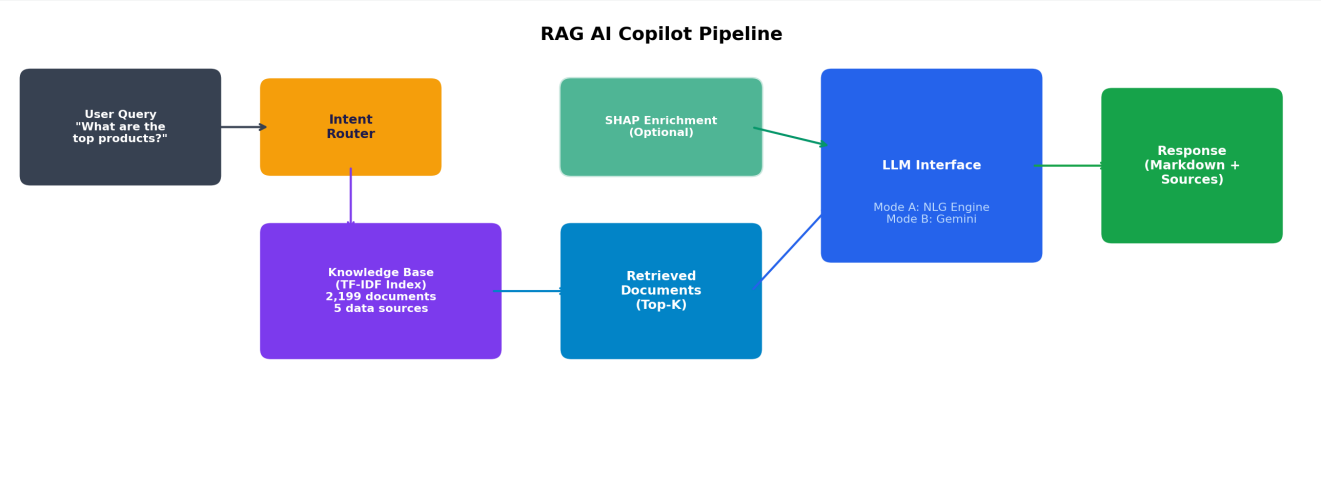


Figure 9.1 - RAG AI Copilot pipeline: query, retrieve, augment, generate

The RAG pipeline (src/genai/rag_pipeline.py) follows a four-stage process. First, the user's natural language query is received and preprocessed. Second, the knowledge base retrieves the top-K most relevant documents using TF-IDF cosine similarity, augmented with keyword matching for exact term queries. Third, the retrieved documents and the original query are assembled into a structured prompt that provides the LLM with the necessary context. Fourth, the Google Gemini LLM generates a natural language response that directly addresses the user's question while grounding its answers in the retrieved knowledge base documents.

ch9_sec3319.4 NLG Engine and Intent Routingch9_sec339.4 NLG Engine and Intent Routing

The LLM interface (src/genai/llm_interface.py) implements a natural language generation engine with seven intent routers that classify the user's query into one of seven categories: forecasting questions, inventory status, model explanation, feature information, system architecture, general knowledge, and data statistics. Each intent router uses keyword patterns and semantic matching to determine the query's intent, which guides the knowledge base retrieval strategy and shapes the prompt template sent to the LLM. This intent-based routing ensures that responses are focused, relevant, and leverage the most appropriate knowledge base sources for each query type.

The Gemini integration uses the Google Generative AI SDK to communicate with the Gemini Pro model. The LLM is prompted with a system message that establishes its role as a retail analytics expert, followed by the retrieved knowledge base context and the user's question. Temperature is set to 0.3 for factual, consistent responses. The system implements fallback handling: if the LLM service is unavailable, the system returns the top retrieved documents directly, ensuring that users always receive useful information even during service disruptions.

ch10Chapter 10: FastAPI Backend (Module 8)

ch10Chapter 10: FastAPI Backend (Module 8)

ch10_sec34110.1 API Architecturech10_sec3410.1 API Architecture

The FastAPI backend (`src/api/main.py`) is the central integration layer that exposes all platform capabilities through a RESTful API. It serves as the bridge between the machine learning models and the frontend dashboard, handling request validation, model inference, data aggregation, and response serialization. The backend implements 11 endpoints organized into logical groups: model operations, forecasting, explainability, inventory, analytics, and the AI Copilot.

ch10_sec35110.2 Lazy Loading and AppStatech10_sec3510.2 Lazy Loading and AppState

A key architectural decision is the use of lazy loading for all heavy ML models and data artifacts. The `AppState` class serves as a singleton container that initializes all models, data, and utilities on first access rather than at application startup. This design reduces cold-start time from over 30 seconds (loading all models and data) to under 2 seconds (creating the FastAPI app), which is critical for serverless deployments and containerized environments where startup latency directly impacts user experience.

The `AppState` class holds references to the trained models (LightGBM, XGBoost, CatBoost), the engineered features `DataFrame`, the processed sales data, the SHAP explainers, the feature list, and the RAG pipeline components. A global `app_state` instance is created at module level, and each endpoint accesses it via dependency injection or direct reference. Thread safety is ensured through Python's GIL for read-only model inference operations.

ch10_sec36110.3 Pydantic Models and CORSch10_sec3610.3 Pydantic Models and CORS

All API requests and responses are validated using Pydantic models, which provide automatic type checking, serialization, and OpenAPI schema generation. Request models include `ForecastRequest` (`store_id`, `item_id`, `days`), `ExplainRequest` (`store_id`, `item_id`, `date`), and `ChatRequest` (`message`, `context`). Response models wrap the result data with status fields and metadata. The CORS middleware is configured to allow cross-origin requests from the Next.js development server, enabling seamless local development with separate frontend and backend processes.

ch10_sec37110.4 Endpoint Referencech10_sec3710.4 Endpoint Reference

Method	Path	Description
GET	/api/health	Health check and service status
GET	/api/models	List available models with performance metrics
POST	/api/forecast	Generate demand forecast for a store-item pair
POST	/api/explain	Get SHAP explanation for a specific prediction
GET	/api/features/importance	Global SHAP feature importance ranking
POST	/api/inventory/recommend	Generate inventory recommendations for a store-item pair
GET	/api/inventory/summary	Aggregate inventory status across all store-items
POST	/api/chat	Send a query to the RAG AI Copilot
GET	/api/analytics/stores	Store-level sales performance analytics
GET	/api/analytics/categories	Category distribution and product counts
GET	/api/data/selectors	Available stores, items, and date ranges

ch110Chapter 11: Frontend Dashboard (Module 9)ch11Chapter 11: Frontend Dashboard (Module 9)

ch11_sec3811.1 Technology and Frameworkch11_sec3811.1 Technology and Framework

The frontend dashboard is a single-page application (SPA) built with Next.js 16 and React 19, using plain JavaScript (JSX) for all dashboard components. Next.js 16 provides the App Router architecture with server-side rendering capabilities, although the dashboard itself is entirely client-rendered to support real-time interactivity. The application uses a tab-based layout implemented with shadcn/ui Tabs components, providing six primary views that correspond to the platform's core capabilities.

ch11_sec3911.2 Dashboard Tabsch11_sec3911.2 Dashboard Tabs

The dashboard is organized into six tabs, each providing a distinct view of the platform's data and AI capabilities. The Overview tab displays key performance metrics (total sales, active products, model accuracy) alongside the store performance chart and category distribution visualization. The Forecast tab allows users to select a store and item, then view a 28-day demand forecast with confidence intervals. The Explain tab provides SHAP-based model explanations, including the waterfall plot for individual predictions and the global feature importance ranking.

The Inventory tab shows inventory status for all store-item combinations, with color-coded status badges and reorder recommendations. The Analytics tab provides detailed store performance

comparisons and category breakdowns using dual-axis charts. The AI Copilot tab features a chat interface where users can ask natural language questions and receive contextual responses from the RAG pipeline. Each tab uses TanStack Query for data fetching, providing automatic loading states, error handling, and background refresh capabilities.

ch11_sec40111.3 API Proxy and Data Layerch11_sec4011.3 API Proxy and Data Layer

The frontend communicates with the FastAPI backend through a Next.js API proxy route defined in `src/app/api/[...slug]/route.ts`. This proxy pattern solves two problems: it avoids CORS issues by keeping all requests on the same origin, and it abstracts the backend URL configuration (including the `XTransformPort` query parameter for the gateway) away from the frontend components. The API client (`src/lib/api.js`) provides a clean, typed interface for all backend endpoints, handling request construction, error parsing, and response transformation.

TanStack Query (React Query) manages all server state through custom hooks defined in `query-provider.jsx`. Each tab's data fetching is encapsulated in `useQuery` hooks with appropriate stale times, cache keys, and refetch policies. This architecture ensures that the UI is always consistent with the backend state, automatically retries failed requests, and provides a smooth user experience even under slow network conditions. The data selectors component provides shared store/item dropdowns used across multiple tabs.

ch11_sec41111.4 UI Components and Stylingch11_sec4111.4 UI Components and Styling

The dashboard uses `shadcn/ui` (New York style) as the component library, providing over 50 production-quality, accessible UI primitives. Key components used include `Card` (for metric displays and chart containers), `Table` (for data tables), `Tabs` (for navigation), `Badge` (for status indicators), `Select` (for store/item dropdowns), `Tooltip` (for chart hover information), and `Skeleton` (for loading states). `Framer Motion` provides smooth tab transitions, card hover effects, and staggered list animations that enhance the user experience without impacting performance. The overall design follows a clean, professional aesthetic with a neutral color palette, consistent spacing, and clear visual hierarchy.

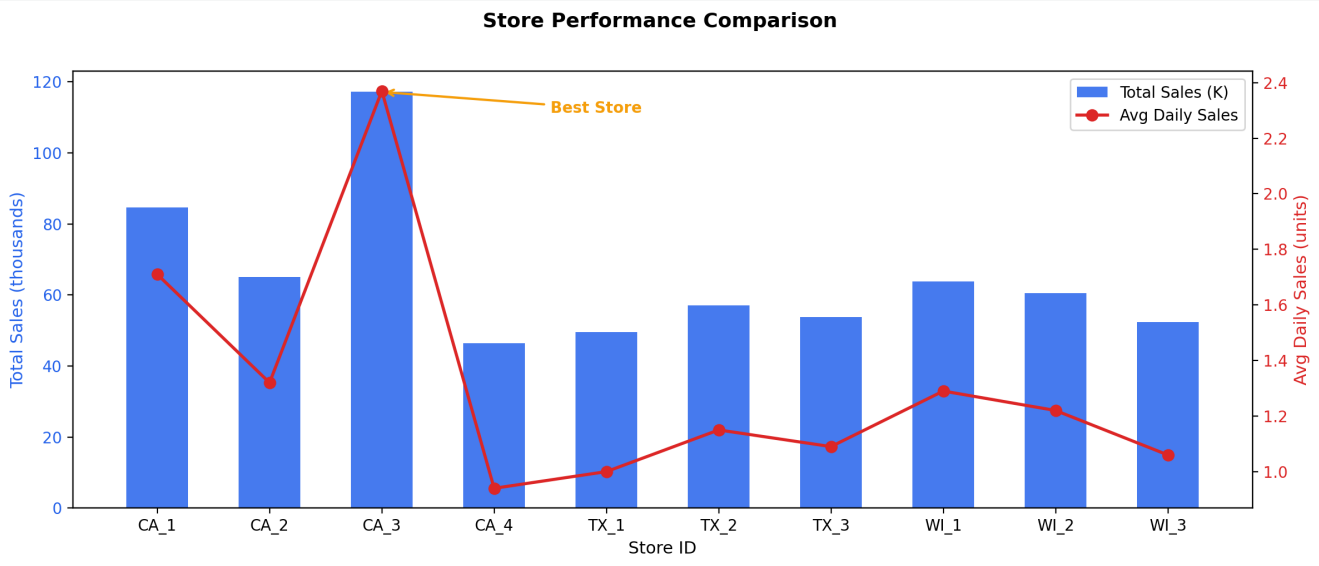


Figure 11.1 - Store sales performance with average daily sales dual-axis chart

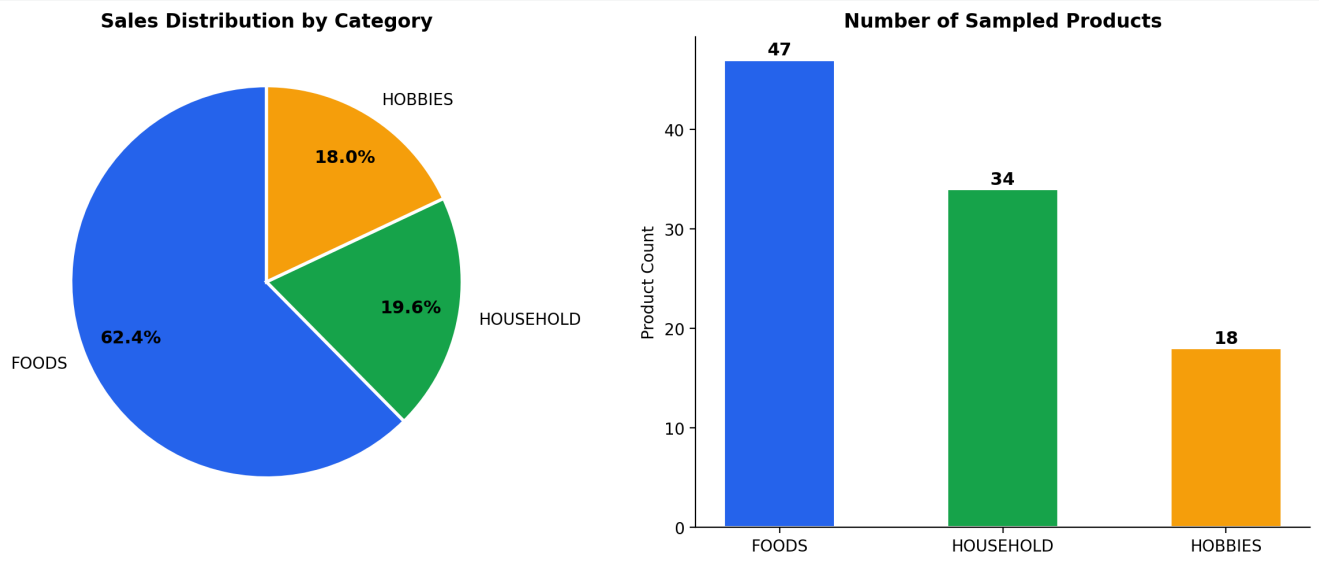


Figure 11.2 - Category distribution pie chart with product count breakdown

ch120Chapter 12: Complete File Reference

Referencech12Chapter 12: Complete File Reference

ch12_sec42112.1 Backend Python Modulesch12_sec4212.1 Backend Python Modules

The following table lists every Python source file in the backend, organized by module. Each entry includes the file path, its purpose, approximate line count, and key connections to other files in the project. The backend consists of seven functional modules plus a shared utilities package.

File Path	Purpose	Lines	Connections
src/data_engineering/__init__.py	Package init	0	None
src/data_engineering/etl.py	ETL pipeline: sampling, melting, optimization	180	Output: processed_sales.parquet
src/feature_engineering/__init__.py	Package init	0	None
src/feature_engineering/features.py	80+ features across 6 groups	346	Input: processed_sales.parquet; Output: engineered_features.parquet
src/models/__init__.py	Package init	0	None
src/models/train.py	Train LightGBM, XGBoost, CatBoost	216	Input: engineered_features.parquet; Output: models/saved/*.pkl
src/explainability/__init__.py	Package init	0	None
src/explainability/shap_explainer.py	SHAP TreeExplainer, local and global	172	Input: models/saved/*.pkl
src/inventory/__init__.py	Package init	0	None
src/inventory/engine.py	Safety stock, reorder, status classification	132	Input: features + model predictions
src/genai/__init__.py	Package init	0	None
src/genai/knowledge_base.py	Build/load 2,199-doc knowledge base	481	Output: kb_docs.json, kb_index.npz
src/genai/rag_pipeline.py	RAG retrieve + generate pipeline	207	Input: knowledge_base; Uses: llm_interface
src/genai/llm_interface.py	Gemini LLM, 7 intent routers	403	Input: rag_pipeline context; External: Gemini API
src/api/__init__.py	Package init	0	None
src/api/main.py	FastAPI app, 11 endpoints, AppState	569	Imports all modules; CORS; lazy loading
src/utils/__init__.py	Package init	0	None
src/utils/config.py	Centralized paths and parameters	92	Used by all modules
scripts/run_pipeline.py	End-to-end pipeline orchestrator	71	Calls: etl, features, train
scripts/lightgbm_xgboost_notebook.py	LightGBM and XGBoost training notebook	467	Calls: train.py functions
scripts/catboost_notebook.py	CatBoost training notebook	327	Calls: train.py functions

File Path	Purpose	Line s	Connections
scripts/inspect_models.py	Model inspection and comparison	81	Loads: models/saved/*.pkl
scripts/notebook_walkthrough.py	Step-by-step walkthrough script	152	Demonstrates all modules

ch12_sec43112.2 Frontend Filesch12_sec4312.2 Frontend Files

The frontend consists of the Next.js application files, dashboard components, shared UI library, and utility modules. The shadcn/ui components in `src/components/ui/` are pre-built library components and are not listed individually. The custom application files are:

File Path	Purpose	Line s	Connections
src/app/layout.tsx	Root layout with providers	48	Wraps page.tsx
src/app/page.tsx	Main dashboard page with tabs	65	Imports all dashboard tabs
src/app/globals.css	Global styles and Tailwind	320+	Imported by layout.tsx
src/app/api/[...slug]/route.ts	API proxy for FastAPI backend	56	Forwards to backend via gateway
src/lib/api.js	API client for all backend endpoints	109	Used by all dashboard tabs
src/lib/constants.js	Shared constants and enums	12	Used by dashboard components
src/lib/utils.ts	Utility functions (cn helper)	6	Used by UI components
src/lib/db.ts	Database client (Prisma)	12	Not used in current scope
src/hooks/use-toast.ts	Toast notification hook	193	Used by dashboard components
src/hooks/use-mobile.ts	Mobile viewport detection	19	Used by responsive layouts
src/components/dashboard/overview-tab.jsx	Overview: KPIs, store chart, category pie	153	Uses: api.js, Recharts, shadcn
src/components/dashboard/forecast-tab.jsx	28-day demand forecast display	178	Uses: api.js, data-selectors
src/components/dashboard/explain-tab.jsx	SHAP waterfall and feature importance	152	Uses: api.js, Recharts
src/components/dashboard/inventory-tab.jsx	Inventory status table and recommendations	164	Uses: api.js, shadcn Table
src/components/dashboard/analytics-tab.jsx	Store analytics and category breakdowns	177	Uses: api.js, Recharts
src/components/dashboard/copilot-tab.jsx	AI Copilot chat interface	189	Uses: api.js /chat endpoint

File Path	Purpose	Line s	Connections
src/components/dashboard/data-selectors.jsx	Shared store/item dropdown selectors	59	Used by: forecast, explain, inventory
src/components/dashboard/query-provider.jsx	TanStack Query provider setup	13	Wraps all dashboard tabs

ch12_sec4412.3 Configuration and Infrastructure Files

The project includes several configuration and infrastructure files that define dependencies, containerization, deployment, and development settings.

File	Description	Lines
requirements.txt	Python dependencies (pandas, lightgbm, xgboost, catboost, shap, fastapi, etc.)	11
Dockerfile	Multi-stage Docker build for backend service	11
start.sh	Startup script for both backend and frontend services	39
package.json	Node.js dependencies (next, react, recharts, framer-motion, tanstack-query, etc.)	94
next.config.ts	Next.js 16 configuration (App Router, transpilation)	12
components.json	shadcn/ui component library configuration	1
tailwind.config.ts	Tailwind CSS 4 theme and plugin configuration	25
postcss.config.mjs	PostCSS configuration for Tailwind	5
tsconfig.json	TypeScript compiler configuration	20
eslint.config.mjs	ESLint linting rules	15